Librería Audit Log Reactive

- Introducción
- Características
- Prerequisitos
- Dependencias
- Configuración
- Implementación en Microservicios

Introducción

La librería **mdw-dm-lib-audit-log-reactive** es un artefacto utilitario que permite a los desarrolladores ahorrar tiempo y recursos en el proceso de implementación de logs.

Características

- Librería agnóstica a Spring boot 3.4.2.
- Compilada en versión Java 21.
- Usa el paradigma de programación Reactiva (Spring Webflux).

Prerequisitos

Antes de agregar la dependencia de la librería **mdw-dm-lib-audit-log-reactive** al proyecto, se debe asegurar que el proyecto cuente con los siguientes requisitos:

- Plugin Spring boot versión 3.4.2 o posterior.
- Compilado en versión Java 21.
- Use el paradigma de programación Reactiva (Spring Webflux).

Dependencias

Copiar la siguiente dependencia según el tipo de compilación:

Gradle

```
implementation 'com.pichincha.li
```

Maven

```
<dependency>
  <groupId>com.pichincha.lib</gr
  <artifactId>mdw-dm-lib-audit-l
  <version>1.0.6</version>
</dependency>
```

Configuración

1. Configuración del archivo application.yml

```yaml
spring:
  kafka:
    properties:
      sasl:
        mechanism: PLAIN
        jaas:
          config: org.apache.kafka.common.security.plain.PlainLoginModu
      bootstrap:
        servers: ${CONFLUENT-HOST-NOPROD}
      security:
        protocol: SASL_SSL
      session:
        timeout:
          ms: 45000
      request:
        timeout:
          ms: 2000
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSeria
      value-serializer: org.apache.kafka.common.serialization.StringSer

logging:
  auditor:
    mode: 'EXTERNAL'
    kafka:
      topic:
        name: ${KAFKA_TOPIC_AUDITOR}
    obfuscation:
      fields: ${MY_FIELDS_TO_OBFUSCATE:}
      enable: false
    executor:
      isDefault: false
      corePoolSize: ${THREAD_CORE_POOL_SIZE}
      maxPoolSize: ${THREAD_MAX_POOL_SIZE}
      keepAliveTime: ${THREAD_KEEP_ALIVE_TIME}
      queueSize: ${THREAD_QUEUE_SIZE}
  step:
    aspect:
      mode: ${LOGGING_STEP_ASPECT_MODE}

application:
  cache:
    max-size: 500
    expire-time-minutes: 10

control:
  error:
    trace: false
```

2. Atributos:

   2.1 Kafka:

Campo	Requerido	Tipo	Default	Descripción
Variable de ambiente				

`spring.kafka.properties.sasl.mechanism`	**SI**	***String***	`PLAIN`	Mecanismo SASL para Kafka
`spring.kafka.properties.sasl.jaas.config`	**SI**	***String***	`org.apache.kafka.common.security.plain.PlainLoginModule required username="${CONFLUENT-KEY-NOPROD}" password="${CONFLUENT-SECRET-NOPROD}"`	Configuración de Jaas para Kafka, se debe especificar el key y secret kafka acorde al ambiente de despliegue
`spring.kafka.properties.bootstrap.servers`	**SI**	***String***	`${CONFLUENT-HOST-NOPROD}`	Servidores de inicio de Kafka
`spring.kafka.properties.security.protocol`	**SI**	***String***	`SASL_SSL`	Protocolo de seguridad de Kafka
`spring.kafka.properties.session.timeout.ms`	**SI**	***Number***	`45000`	Tiempo de espera de sesión para Kafka
`spring.kafka.properties.request.timeout.ms`	**SI**	***Number***	`2000`	Tiempo de espera para una Solicitud a Kafka
`spring.kafka.producer.key-serializer`	**SI**	***String***	`org.apache.kafka.common.serializa`	Clase serializadora de claves

			`tion.String Serializer`	para el productor de Kafka
`spring.kafka.producer.value-serializer`	**SI**	***String***	`org.apache.kafka.common.serialization.String Serializer`	Clase de serializador de valor para el productor de Kafka

2.2 Logging:

Campo Variable de ambiente	Requerido	Tipo	Default	Descripción
`logging.auditor.mode`	**SI**	***String***	`EXTERNAL`	Modo para el registro del auditor
`logging.auditor.kafka.topic.name`	**SI**	***String***	`${KAFKA_TOPIC_AUDITOR}`	Tópico de Kafka para registrar eventos de auditoría
`logging.auditor.obfuscation.fields`	**SI**	***String***	`${MY_FIELDS_TO_OBFUSCATE:}`	Campos a ofuscar en los registros
`logging.auditor.obfuscation.enable`	**SI**	***Boolean***	`false`	Habilitar la ofuscación de campos
`logging.auditor.executor.isDefault`	**SI**	***Boolean***	`false`	Bandera para indicar si el ejecutor debe utilizar la configuració

				n predetermin ada
`logging.auditor.executor.corePoolSize`	**SI**	***Number***	`${THREAD_CORE_POOL_SIZE}`	Tamaño del grupo de núcleos para el ejecutor del auditor de registro
`logging.auditor.executor.maxPoolSize`	**SI**	***Number***	`${THREAD_MAX_POOL_SIZE}`	Tamaño máximo del grupo para el ejecutor del auditor de registro
`logging.auditor.executor.keepAliveTime`	**SI**	***Number***	`${THREAD_KEEP_ALIVE_TIME}`	Tiempo de vida para el ejecutor
`logging.auditor.executor.queueSize`	**SI**	***Number***	`${THREAD_QUEUE_SIZE}`	Tamaño de la cola para el ejecutor
`logging.step.aspect.mode`	**SI**	***String***	`${LOGGING_STEP_ASPECT_MODE}`	Bandera para identificar el modo de registro de respuesta

Implementación en Microservicios

1. Detalle de los componentes de la librería:

Nombre	Descripción
`@LogAudit`	Es una anotación personalizada que se utiliza en los controladores que

	exponen endpoints y sirve para auditar las llamadas al método donde está aplicada, indica que cada vez que se invoque este endpoint, se registrará información relevante para auditoría, como parámetros de entrada, salida, o detalles de la ejecución.
`@LogAuditStep`	Es una anotación personalizada que se utiliza en los servicios, adaptadores, implementaciones, etc., al aplicar esta anotación, se puede interceptar la ejecución del método para agregar lógica de auditoría, guardar logs detallados y rastrear el flujo de la operación. Esto es útil para monitoreo y trazabilidad.

2. Implementación
   a. Añadir la dependencia al proyecto.

> *Anotar el controlador para el flujo principal, anotar el servicio implementación o adaptador como hijo del flujo principal*

Ejemplo consumo uso

```
@RestController
@RequiredArgsConstructor
public class CreditCardController implements BusinessCapabilityApi {

    private final CreditCardService creditCardService;
    private final CreditCardResponseMapper responseMapper;

    @Override
    @LogAudit
    public Mono<ResponseEntity<PostCreditCardCustomerPositionRetrieveBy
            String xApp, String xGuid, String xChannel, String xMedium,
            String xDeviceIp, String xAgency, String xGeolocation, Inte
            PostCreditCardCustomerPositionRetrieveByFilter postCreditCa
            ServerWebExchange exchange) {
        return creditCardService.retrieveCreditCards(requestMapper.toDo
                .flatMapSequential(domain -> Mono.fromCallable(() ->
                        responseMapper.toResponse(domain)).subscribeOn(
                .map(ResponseEntity::ok);
    }
}
```

```
@Service
@RequiredArgsConstructor
public class CreditCardServiceAdapter implements CreditCardService {

    private final CreditCardStrategy strategy;
    private final CardPaymentAgreementPort cardPaymentAgreementPort;

    @Override
    @LogAuditStep
    public Flux<CreditCard> retrieveCreditCards(CreditCardRequest reque
        var accountRules = rulesPort.retrieveCreditCardRules().cache();
        return retrieveChannelAttributes().flatMapMany(attributes -> ca
                .validateCustomerCreditCard(request.cardAgreementIdenti
                .thenMany(Flux.deferContextual(ctx -> strategy
                        .retrieveCreditCards(request.retrievedCreditCar
                        .flatMapSequential(creditCard -> applyCreditCar
                .switchIfEmpty(Flux.defer(() -> Flux.error(new GlobalEr
                        getClass().getSimpleName(), HttpStatus.NOT_FOUN
    }
}
```